

LEVEL 4 PROJECT REPORT

Jenkins CI/CD Pipeline for a Java Web Application

Using GitHub + Jenkins + Apache Maven + AWS EC2 + Apache Tomcat

AWS Region: Asia Pacific (Mumbai) — ap-south-1

Technology / Service	Role in the Project
GitHub	Stores the Java application source and CI/CD project files.
Jenkins	Coordinates checkout, build and deployment stages.
Apache Maven	Compiles/tests/packages the application and produces the WAR artifact.
Java	Provides the Java environment used by the build and Tomcat application.
AWS EC2	Hosts the Jenkins environment and the Tomcat application server.
Apache Tomcat	Runs and serves the packaged Java web application.
SSH / SCP	Supports Jenkins-to-Tomcat server communication and WAR delivery.
Jenkins Credentials	Stores credentials/keys used by GitHub and Tomcat deployment connections.

Prepared by Manimaran

Implementation evidence: 25 screenshots + 1 system architecture diagram

Table of Contents

1. Project Overview
 2. Problem Statement
 3. Objectives
 4. Technologies and Tools
 5. Prerequisites
 6. System Architecture
 7. Complete CI/CD Workflow
 8. Project Components
 9. Implementation Procedure
 10. Testing and Validation
 11. Troubleshooting and Operational Notes
 12. Security Considerations
 13. Advantages
 14. Limitations
 15. Future Enhancements
 16. Key Lessons Learned
 17. Interview / Viva Explanation
 18. Conclusion
- Appendix A — Implementation Evidence Inventory

1. Project Overview

This Level 4 project demonstrates a Continuous Integration and Continuous Deployment (CI/CD) workflow for a Java web application using GitHub, Jenkins, Apache Maven and Apache Tomcat on AWS EC2. The supplied architecture shows a developer pushing source code to GitHub, GitHub notifying Jenkins through a webhook, Jenkins checking out the project and invoking Maven for build/test/package activities, and the resulting WAR application being transferred to a separate Tomcat EC2 server through SSH/SCP. The deployed application is then accessed by a user over HTTP on port 8080.

The implementation evidence supplied for this report shows the Jenkins dashboard and pipeline stages, Maven/Java environment checks, Tomcat installation and service management, Tomcat Manager configuration, Jenkins credentials, the Jenkins workspace, successful pipeline execution, and browser-level validation of the SampleApp.

2. Problem Statement

Manual Java web application deployment requires repeated source retrieval, compilation, WAR packaging, artifact transfer and application-server restart/verification activities. Repeating these tasks manually can increase deployment effort and introduce configuration mistakes. This project addresses that workflow by using Jenkins to coordinate source checkout and build activities and by automating delivery of the WAR artifact to the Tomcat EC2 application server.

3. Objectives

- Maintain the Java web application source code in GitHub.
- Use Jenkins as the CI/CD orchestration server.
- Use Apache Maven to compile, test and package the Java application.
- Generate a deployable WAR artifact.
- Use SSH/SCP-based communication for transferring the application artifact to the Tomcat EC2 host.
- Run the Java web application using Apache Tomcat.
- Expose the application through the Tomcat HTTP endpoint on port 8080.
- Provide visible Jenkins stages and console output for pipeline verification.
- Validate the deployed application through the Tomcat Manager and browser.

4. Technologies and Tools

Technology / Service	Purpose	Role in the Project
GitHub	Source control	Stores the Java application source and CI/CD project files.
Jenkins	CI/CD automation	Coordinates checkout, build and deployment stages.
Apache Maven	Java build automation	Compiles/tests/packages the application and produces the WAR artifact.
Java	Application build/runtime	Provides the Java environment used by the build and Tomcat application.
AWS EC2	Compute	Hosts the Jenkins environment and the Tomcat application server.
Apache Tomcat	Java web server	Runs and serves the packaged Java web application.
SSH / SCP	Secure remote transfer	Supports Jenkins-to-Tomcat server communication and WAR delivery.
Jenkins Credentials	Secret management	Stores credentials/keys used by GitHub and Tomcat deployment connections.

5. Prerequisites

- An AWS account with access to EC2 and required security-group configuration.
- A Jenkins server running on an EC2 instance.
- A separate EC2 instance configured with Apache Tomcat.
- Java and Maven available for the Java build process.
- Git and a GitHub repository containing the Java web application.
- A valid pom.xml for Maven packaging.
- SSH connectivity from Jenkins to the Tomcat server.
- Network access to Jenkins and Tomcat on the required ports.

6. System Architecture

The supplied architecture contains two main EC2 responsibilities: a Jenkins/build environment and a Tomcat application server. The source-to-runtime path is shown below.

CI/CD Pipeline System Architecture

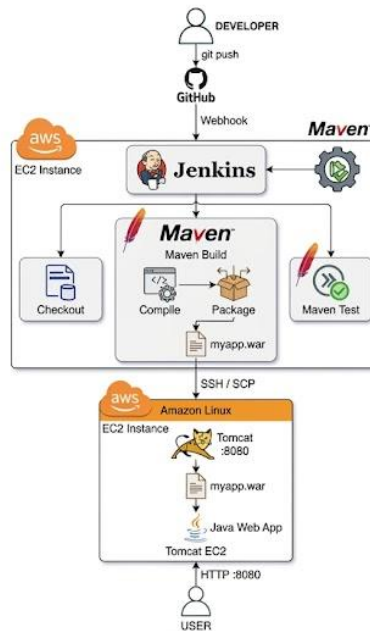


Figure 1. Complete Level 4 CI/CD architecture supplied for the project.

6.1 Architecture Flow

Step	Flow	Purpose
1	Developer → GitHub	The developer pushes application changes to the source repository.
2	GitHub → Jenkins	A webhook notifies Jenkins of the repository event.
3	Jenkins → Checkout	Jenkins retrieves the project source.
4	Jenkins → Maven	Maven performs the Java build/test/package workflow.
5	Maven → WAR	The build produces the deployable WAR artifact.
6	Jenkins → SSH/SCP → Tomcat EC2	The deployment connection transfers the application artifact to the application server.
7	Tomcat EC2 → Tomcat :8080	Tomcat runs the deployed Java web application.
8	User → HTTP :8080	The user accesses the application through the EC2 endpoint.

7. Complete CI/CD Workflow

1. Developer commits and pushes the Java application source to GitHub.
2. GitHub sends the configured webhook event to Jenkins.
3. Jenkins starts the tomcat_deploy-webapp pipeline job.
4. Jenkins checks out the source code into its workspace.
5. Jenkins loads the required build tools and environment.
6. Maven compiles/tests/packages the Java project and creates the WAR artifact.
7. The deployment stage uses the configured SSH connection to reach the Tomcat EC2 server.
8. The WAR is copied/deployed into the Tomcat webapps environment.
9. Tomcat runs the application on port 8080.
10. Jenkins records the execution result and console output.
11. The user validates the deployed application through Tomcat Manager and the browser.

8. Project Components

GitHub Repository: Source repository for the Java web application and pipeline-related files.

Jenkins Pipeline: The tomcat_deploy-webapp job shown in the evidence orchestrates Tool Install, Checkout, Build and Deploy stages.

pom.xml: Maven project/build descriptor used to define the Java build and packaging process.

Jenkins Workspace: Contains the checked-out source, target directory, pom.xml and README.md as shown in the workspace evidence.

Tomcat EC2 Server: Separate application host running Apache Tomcat 10.1.59 and serving the web application.

Tomcat Manager: Web interface used to inspect running applications and validate /sample deployment.

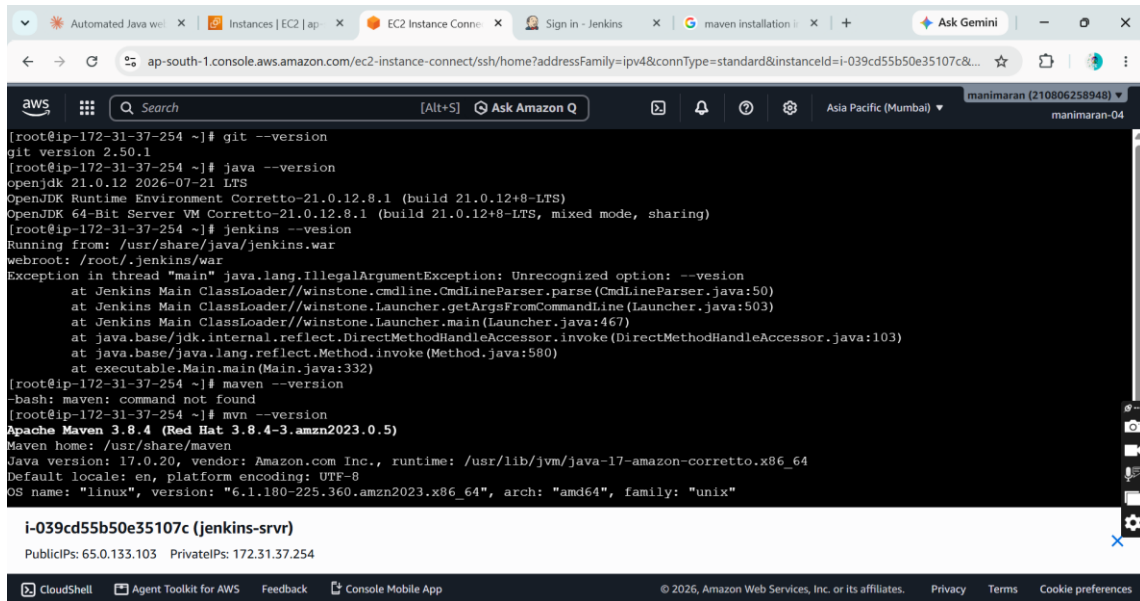
SSH Deployment Credentials: Jenkins credentials used for the Jenkins-to-Tomcat connection; sensitive values are not reproduced in this report.

9. Implementation Procedure

The following 25 steps are presented in the chronological order. Each instruction/action is placed before its corresponding image.

Step 1 – Create Instance & Verify the basic build environment

Instruction / Action: The EC2 terminal verifies Git, Java and Maven availability. The screenshot also records command-level checks performed while preparing the Jenkins build environment.



```

[root@ip-172-31-37-254 ~]# git --version
git version 2.50.1
[root@ip-172-31-37-254 ~]# java --version
openjdk 21.0.2 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)
[root@ip-172-31-37-254 ~]# jenkins --version
Running from: /usr/share/java/jenkins.war
webroot: /root/.jenkins/war
Exception in thread "main" java.lang.IllegalArgumentException: Unrecognized option: --vesion
    at Jenkins Main ClassLoader//winstone.cmdline.CmdLineParser.parse(CmdLineParser.java:50)
    at Jenkins Main ClassLoader//winstone.Launcher.getArgsFromCommandLine(Launcher.java:503)
    at Jenkins Main ClassLoader//winstone.Launcher.main(Launcher.java:467)
    at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:103)
    at java.base/java.lang.reflect.Method.invoke(Method.java:580)
    at executable.Main.main(Main.java:332)
[root@ip-172-31-37-254 ~]# mvn --version
-bash: mvn: command not found
[root@ip-172-31-37-254 ~]# mvn --version
Apache Maven 3.8.4 (Red Hat 3.8.4-3.amzn2023.0.5)
Maven home: /usr/share/maven
Java version: 17.0.20, vendor: Amazon.com Inc., runtime: /usr/lib/jvm/java-17-amazon-corretto.x86_64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.1.180-225.360.amzn2023.x86_64", arch: "amd64", family: "unix"
  
```

Figure 2. Create Instance & Verify the basic build environment.

Step 2 – Verify Jenkins availability and create the pipeline job

Instruction / Action: The Jenkins dashboard is accessible and creating the demo job for checking the Jenkins works correctly.

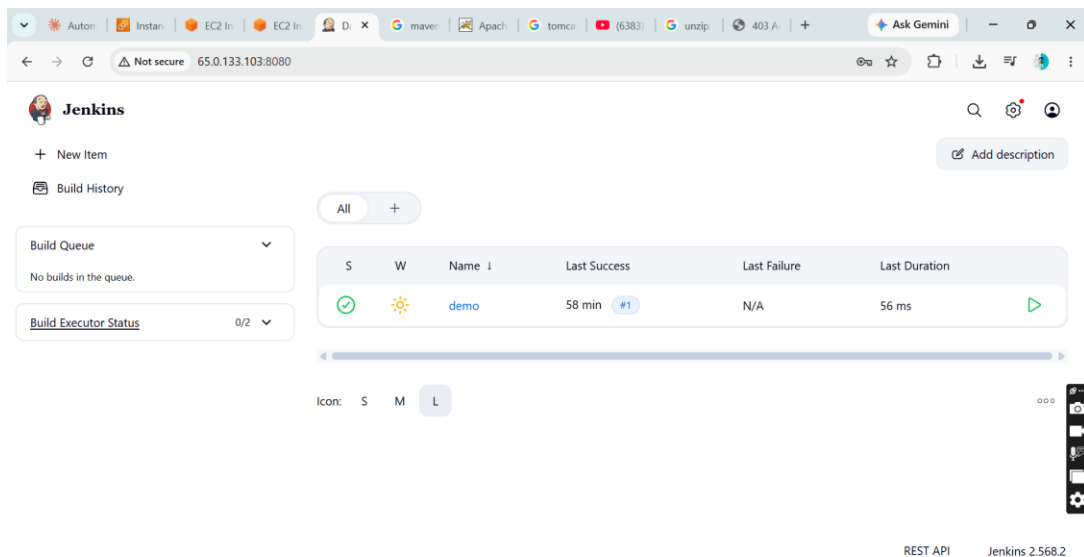
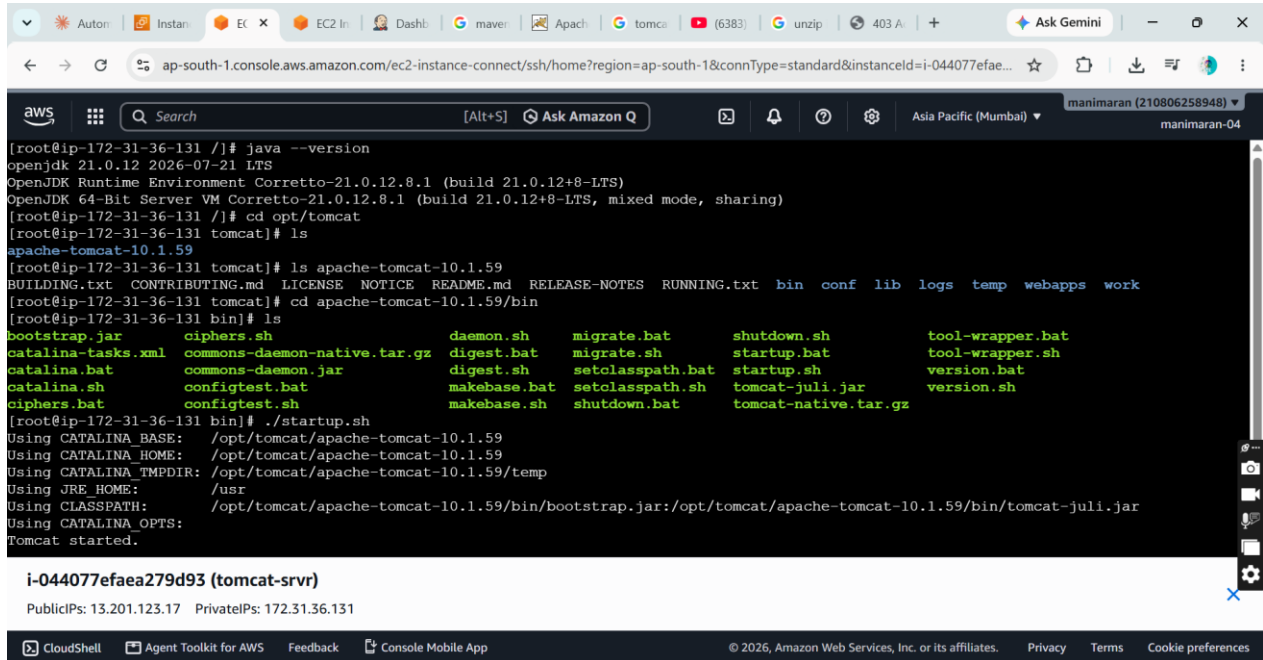


Figure 3. Verify Jenkins availability and create the pipeline job.

Step 3 – Verify Java and Tomcat installation on the application EC2 instance

Instruction / Action: The application server terminal confirms Java is available and shows the Apache Tomcat installation under /opt/tomcat.



```

[root@ip-172-31-36-131 ~]# java --version
openjdk 21.0.12 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)
[root@ip-172-31-36-131 ~]# cd /opt/tomcat
[root@ip-172-31-36-131 tomcat]# ls
apache-tomcat-10.1.59
[root@ip-172-31-36-131 tomcat]# ls apache-tomcat-10.1.59
BUILDING.txt  CONTRIBUTING.md  LICENSE  NOTICE  README.md  RELEASE-NOTES  RUNNING.txt  bin  conf  lib  logs  temp  webapps  work
[root@ip-172-31-36-131 tomcat]# cd apache-tomcat-10.1.59/bin
[root@ip-172-31-36-131 bin]# ls
bootstrap.jar  ciphers.sh  commons-daemon-native.tar.gz  daemon.sh  digest.bat  migrate.bat  shutdown.sh  tool-wrapper.bat
catalina-tasks.xml  commons-daemon.jar  configtest.bat  digest.sh  migrate.sh  shutdown.bat  startup.bat  tool-wrapper.sh
catalina.bat  commons-daemon.jar  makebase.bat  setclasspath.bat  startup.sh  version.bat
catalina.sh  configtest.bat  makebase.bat  setclasspath.sh  tomcat-juli.jar  version.sh
ciphers.bat  configtest.sh  makebase.sh  shutdown.bat  tomcat-native.tar.gz
[root@ip-172-31-36-131 bin]# ./startup.sh
Using CATALINA_BASE:   /opt/tomcat/apache-tomcat-10.1.59
Using CATALINA_HOME:   /opt/tomcat/apache-tomcat-10.1.59
Using CATALINA_TMPDIR: /opt/tomcat/apache-tomcat-10.1.59/temp
Using JRE_HOME:        /usr
Using CLASSPATH:        /opt/tomcat/apache-tomcat-10.1.59/bin/bootstrap.jar:/opt/tomcat/apache-tomcat-10.1.59/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
  
```

i-044077efae279d93 (tomcat-srvr)
PublicIPs: 13.201.123.17 PrivateIPs: 172.31.36.131

Figure 4. Verify Java and Tomcat installation on the application EC2 instance.

Step 4 – Verify Tomcat is reachable on port 8080

Instruction / Action: The browser displays the Apache Tomcat 10.1.59 welcome page, confirming that the Tomcat server is responding on the EC2 endpoint.

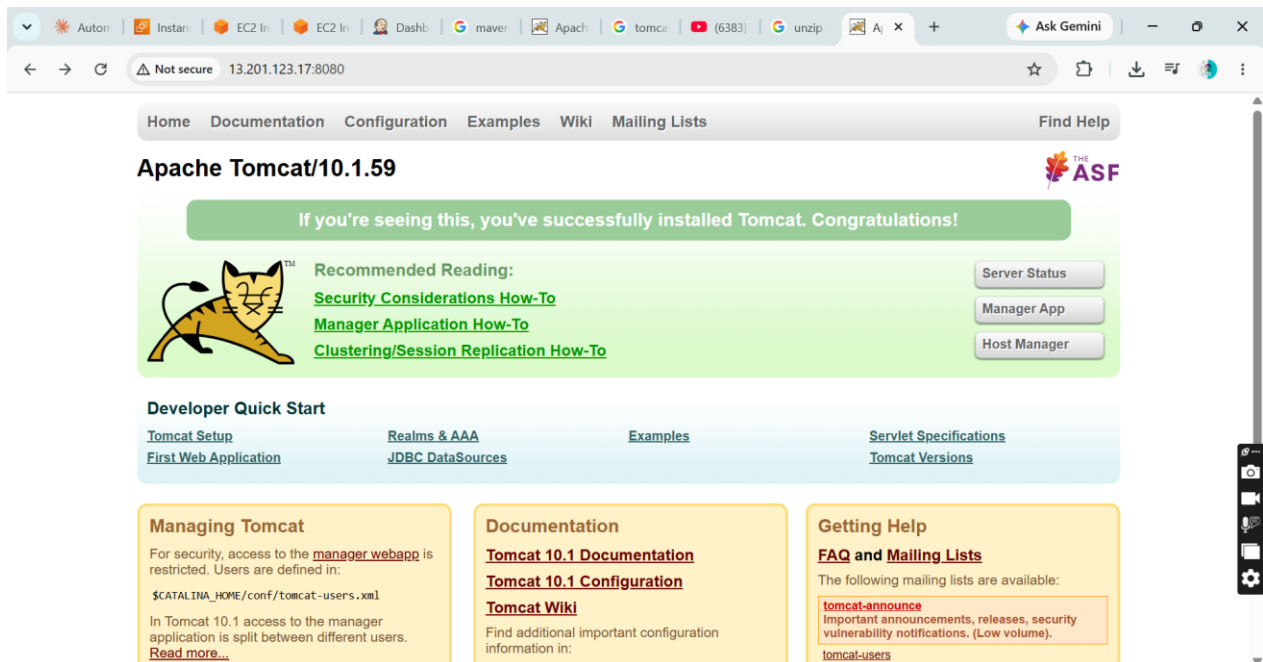


Figure 5. Verify Tomcat is reachable on port 8080.

Step 5 – Attempting to Tomcat Web Application Manager Page

Instruction / Action: The Tomcat Manager defaultly shows permission error (403 Access Denied). Because it allows only peculiar range of cidr ip address to access this page. For accessing this page we need to command that cidr range in context.xml file.

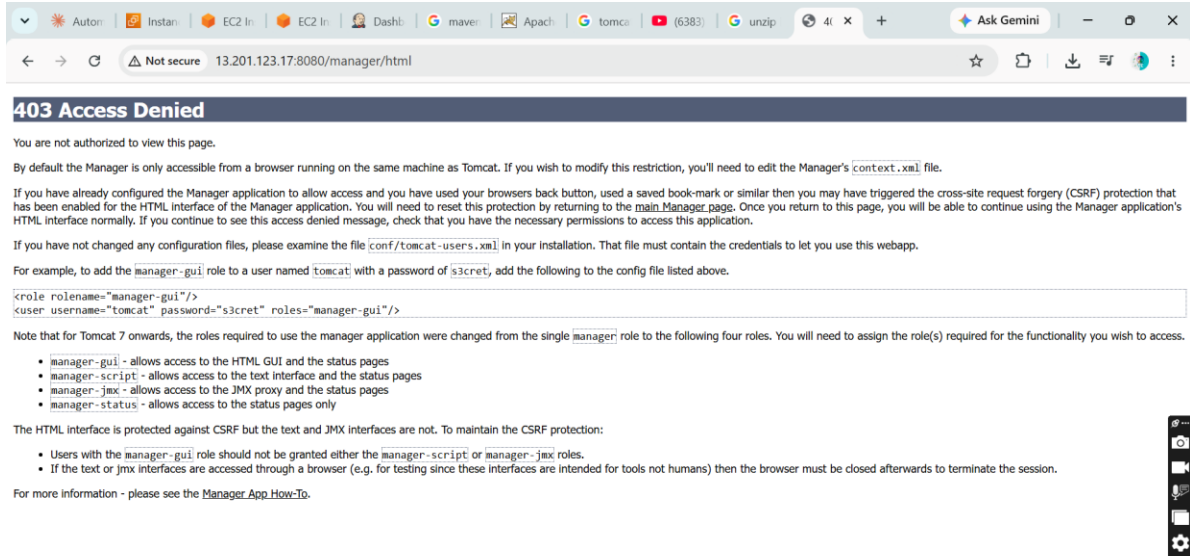


Figure 6. Attempting to Tomcat Web Application Manager Page

Step 6 – Opening the context.xml file

Instruction / Action: Opening the file using vi command. The context.xml file path will be `tomcat-app/webapps/manager/META-INF/context.xml`

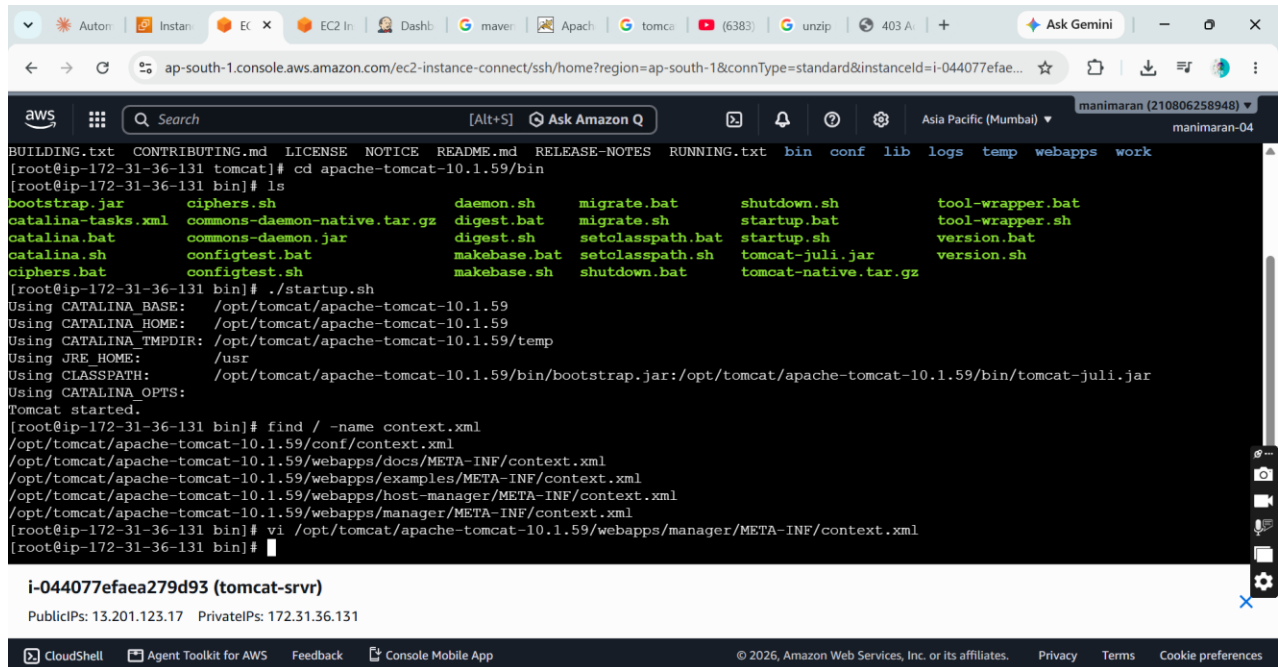


Figure 7. Opening the Context.xml file

Step 7 – Verify the context.xml file

Instruction / Action: Commanding the that peculiar CIDR IP range

```

Licensed to the Apache Software Foundation (ASF) under one or more
contributor license agreements.  See the NOTICE file distributed with
this work for additional information regarding copyright ownership.
The ASF licenses this file to You under the Apache License, Version 2.0
(the "License"); you may not use this file except in compliance with
the License.  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
-->
<Context antiResourceLocking="false" privileged="true" >
  <CookieProcessor className="org.apache.tomcat.util.http.Rfc6265CookieProcessor"
    sameSiteCookies="strict" />
  <!-- <Valve className="org.apache.catalina.valves.RemoteCIDRValve"
    allow="127.0.0.0/8,::1/128" /> -->
  <Manager sessionAttributeValveClassNameFilter="java.lang. (?:Boolean|Integer|Long|Number|String) |org.apache.catalina.filters.CsrfPre
    entionFilter\$LruCache(?:\$1)?|java.util. (?Linked)?HashMap"/>
</Context>
"/opt/tomcat/apache-tomcat-10.1.59/webapps/manager/META-INF/context.xml" [dos] 24L, 1360B      24,10  Bot
i-044077efaea279d93 (tomcat-srvr)
PublicIPs: 13.201.123.17  PrivateIPs: 172.31.36.131
  
```

Figure 8. Verify the context.xml file

Step 8 – Inspect Tomcat configuration and restart the server

Instruction / Action: Tomcat manager and host-manager context configuration files are inspected, followed by Tomcat shutdown/start commands during configuration.

```

/opt/tomcat/apache-tomcat-10.1.59/webapps/examples/META-INF/context.xml
/opt/tomcat/apache-tomcat-10.1.59/webapps/host-manager/META-INF/context.xml
/opt/tomcat/apache-tomcat-10.1.59/webapps/manager/META-INF/context.xml
[root@ip-172-31-36-131 bin]# vi /opt/tomcat/apache-tomcat-10.1.59/webapps/manager/META-INF/context.xml
[root@ip-172-31-36-131 bin]# vi /opt/tomcat/apache-tomcat-10.1.59/webapps/host-manager/META-INF/context.xml
[root@ip-172-31-36-131 bin]# shutdown.sh
-bash: shutdown.sh: command not found
[root@ip-172-31-36-131 bin]# ./shutdown.sh
Using CATALINA_BASE:   /opt/tomcat/apache-tomcat-10.1.59
Using CATALINA_HOME:   /opt/tomcat/apache-tomcat-10.1.59
Using CATALINA_TMPDIR: /opt/tomcat/apache-tomcat-10.1.59/temp
Using JRE_HOME:        /usr
Using CLASSPATH:        /opt/tomcat/apache-tomcat-10.1.59/bin/bootstrap.jar:/opt/tomcat/apache-tomcat-10.1.59/bin/tomcat-juli.jar
[root@ip-172-31-36-131 bin]# ./startup.sh
Using CATALINA_BASE:   /opt/tomcat/apache-tomcat-10.1.59
Using CATALINA_HOME:   /opt/tomcat/apache-tomcat-10.1.59
Using CATALINA_TMPDIR: /opt/tomcat/apache-tomcat-10.1.59/temp
Using JRE_HOME:        /usr
Using CLASSPATH:        /opt/tomcat/apache-tomcat-10.1.59/bin/bootstrap.jar:/opt/tomcat/apache-tomcat-10.1.59/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
[root@ip-172-31-36-131 bin]#
i-044077efaea279d93 (tomcat-srvr)
PublicIPs: 13.201.123.17  PrivateIPs: 172.31.36.131
  
```

Figure 9. Inspect Tomcat configuration and restart the server.

Step 9 – Open the Tomcat Manager application

Instruction / Action: The browser reaches the Tomcat Manager endpoint and presents the authentication dialog required for manager access.

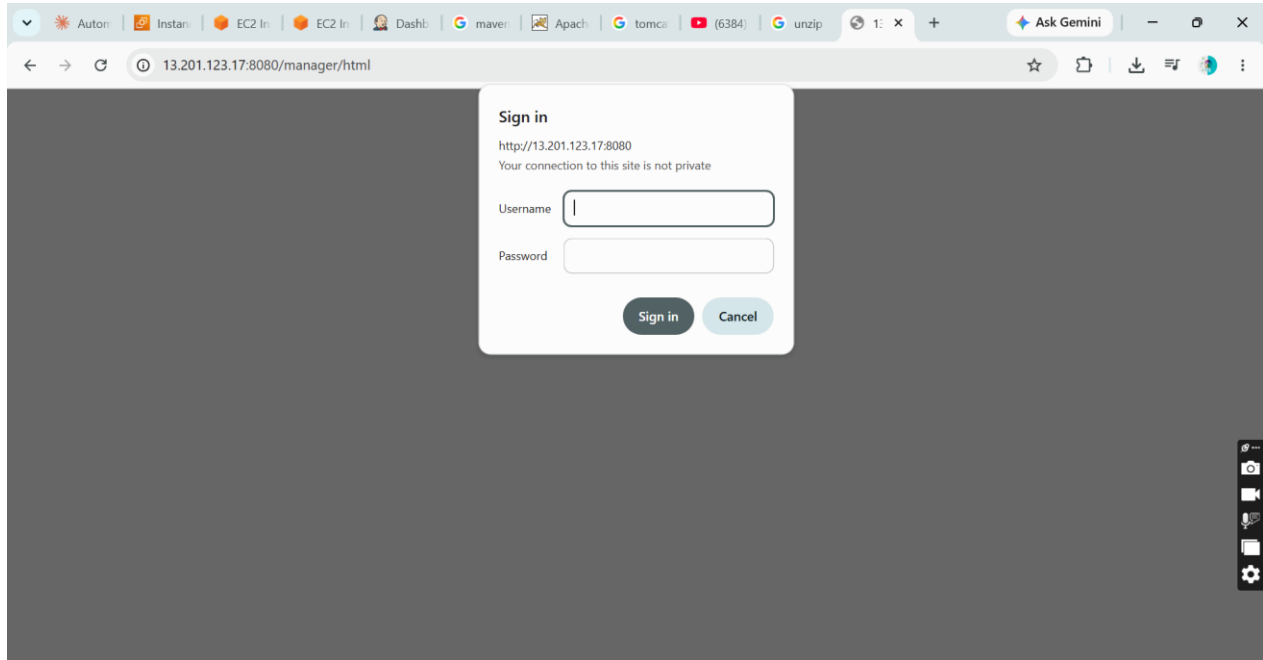


Figure 10. Open the Tomcat Manager application.

Step 10 – Configure Tomcat Manager access

Instruction / Action: The Tomcat user configuration is edited (tomcat-users.xml) to provide the roles required for manager-based deployment. Sensitive credentials have been redacted in this documentation copy.

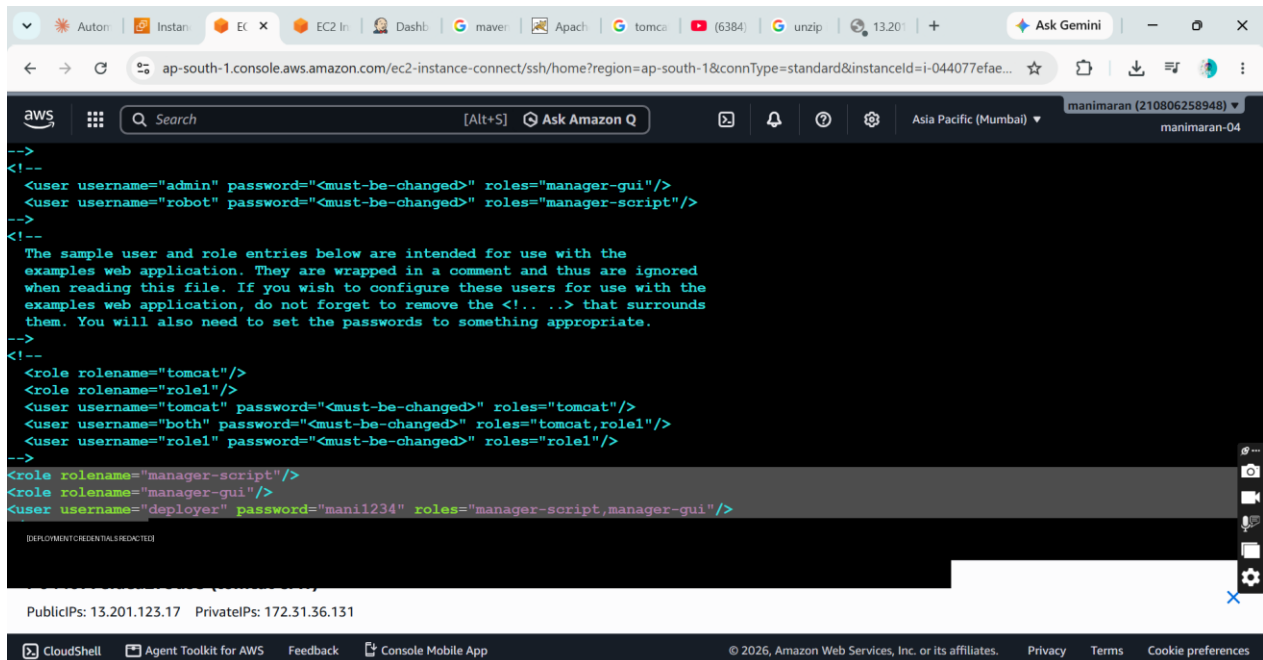


Figure 11. Configure Tomcat Manager access.

Step 11 – Authenticate to Tomcat Manager

Instruction / Action: The Tomcat Manager sign-in page is shown as part of validating the configured manager access.

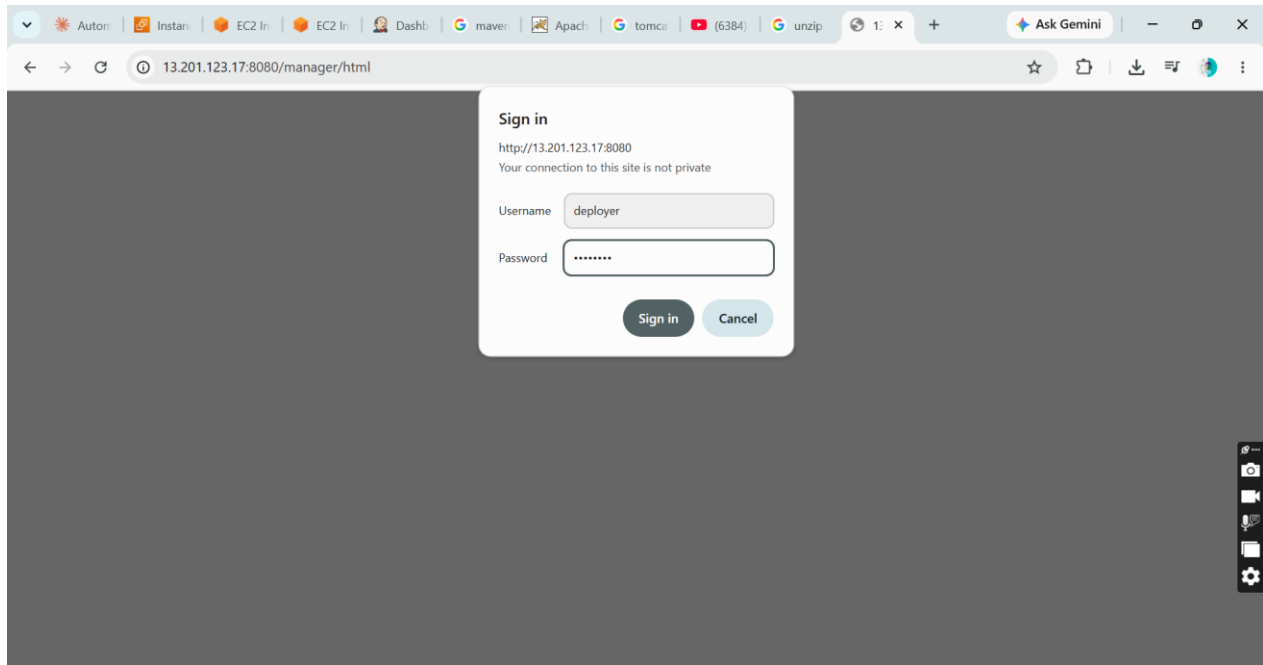


Figure 12. Authenticate to Tomcat Manager.

Step 12 – Verify the Tomcat Web Application Manager

Instruction / Action: The Tomcat Web Application Manager lists the installed applications and shows the manager interface operating successfully.

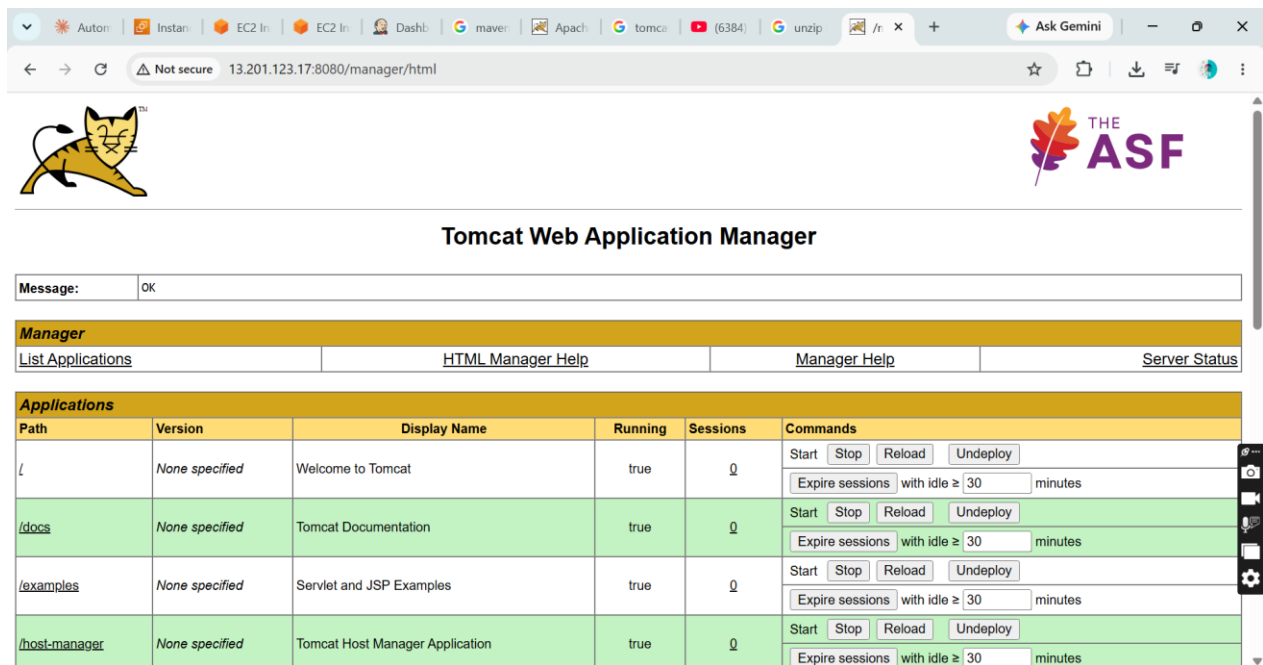


Figure 13. Verify the Tomcat Web Application Manager.

Step 13 – Prepare the Tomcat systemd service

Instruction / Action: The EC2 terminal shows inspection of the system configuration followed by creation/editing of a dedicated tomcat.service unit.

```

aws
Search [Alt+S] Ask Amazon Q
manimaran (210806258948)
manimaran-04

amazon-linux-release    dhcp          hosts          logrotate.conf  pki            selinux         tmpfiles.d
amazon-linux-release-cpe dnf           idmapd.conf    logrotate.d     pm             services        trusted-key.key
asound.conf            dracut.conf   image-id       lsm             popt.d         sestatus.conf  udev
at.deny                dracut.conf.d init.d         machine-id      printcap       sgml            update-motd.d
audit                 environment   inittab        magic           profile.d       shadow          vimrc
bash_completion.d     ethertypes   inputrc        man_db.conf     protocols      shadow-        virc
bashrc                exports       issue          mke2fs.conf    ro.d           shells          wgetrc
bindresvport.blacklist exports.d      issue.net      modprobe.d      rc.d           skel            xattr.conf
binfmt.d              filesystems   java           modules-load.d  rc0.d          ssh             xdg
chkconfig.d           fonts         jvm            motd            rc1.d          ssl             xml
chrony.conf           fstab        jvm-common     mtab            rc2.d          sssd            yum.repos.d
chrony.d              gcrypt       kernel         nanorc          rc3.d          statetab.d
chrony.keys           gnupg        keyutils       netconfig       rc4.d          subgid
cifs-utils            groff        krb5.conf.d    networks        rc5.d          subuid
cloud                 group        ld.so.cache    nfs.conf        rc6.d          request-key.conf
cron.daily             group-

[root@ip-172-31-36-131 etc]# cd ..
[root@ip-172-31-36-131 /]# sudo systemctl status tomcat
o tomcat.service
   Loaded: masked (Reason: Unit tomcat.service is masked.)
   Active: inactive (dead)
[root@ip-172-31-36-131 /]# sudo nano /etc/systemd/system/tomcat.service
[root@ip-172-31-36-131 /]# sudo nano /etc/systemd/system/tomcat.service
[root@ip-172-31-36-131 /]#

i-044077efaea279d93 (tomcat-srvr)
PublicIPs: 13.201.123.17  PrivateIPs: 172.31.36.131

```

Figure 14. Prepare the Tomcat systemd service.

Step 14 – Start Tomcat through systemd and verify service status

Instruction / Action: The tomcat.service unit is reloaded and started. The status output shows Apache Tomcat active with the Java process running successfully.

```

aws
Search [Alt+S] Ask Amazon Q
manimaran (210806258948)
manimaran-04

[root@ip-172-31-36-131 /]# sudo vi /etc/systemd/system/tomcat.service
[root@ip-172-31-36-131 /]# sudo systemctl start tomcat
Warning: The unit file, source configuration file or drop-ins of tomcat.service changed on disk. Run 'systemctl daemon-reload' to reload units.
Job for tomcat.service failed because the control process exited with error code.
See "systemctl status tomcat.service" and "journalctl -xeu tomcat.service" for details.
[root@ip-172-31-36-131 /]# sudo systemctl daemon-reload
[root@ip-172-31-36-131 /]# sudo systemctl start tomcat
[root@ip-172-31-36-131 /]# sudo systemctl status tomcat
● tomcat.service - Apache Tomcat
   Loaded: loaded (/etc/systemd/system/tomcat.service; disabled; preset: disabled)
   Active: active (running) since Fri 2026-08-28 10:19:51 UTC; 15s ago
     Process: 31459 ExecStart=/opt/tomcat/apache-tomcat-10.1.59/bin/startup.sh (code=exited, status=0/SUCCESS)
    Main PID: 31466 (java)
      Tasks: 35 (limit: 4523)
     Memory: 103.0M
        CPU: 3.033s
    CGroup: /system.slice/tomcat.service
            └─31466 /usr/lib/jvm/java-21-amazon-corretto/bin/java -Djava.util.logging.config.file=/opt/tomcat/apache-tomcat-10.1.59/conf/

Aug 28 10:19:51 ip-172-31-36-131.ap-south-1.compute.internal systemd[1]: Starting tomcat.service - Apache Tomcat...
Aug 28 10:19:51 ip-172-31-36-131.ap-south-1.compute.internal startup.sh[31459]: Tomcat started.
Aug 28 10:19:51 ip-172-31-36-131.ap-south-1.compute.internal systemd[1]: Started tomcat.service - Apache Tomcat.
lines 1-14/14 (END)

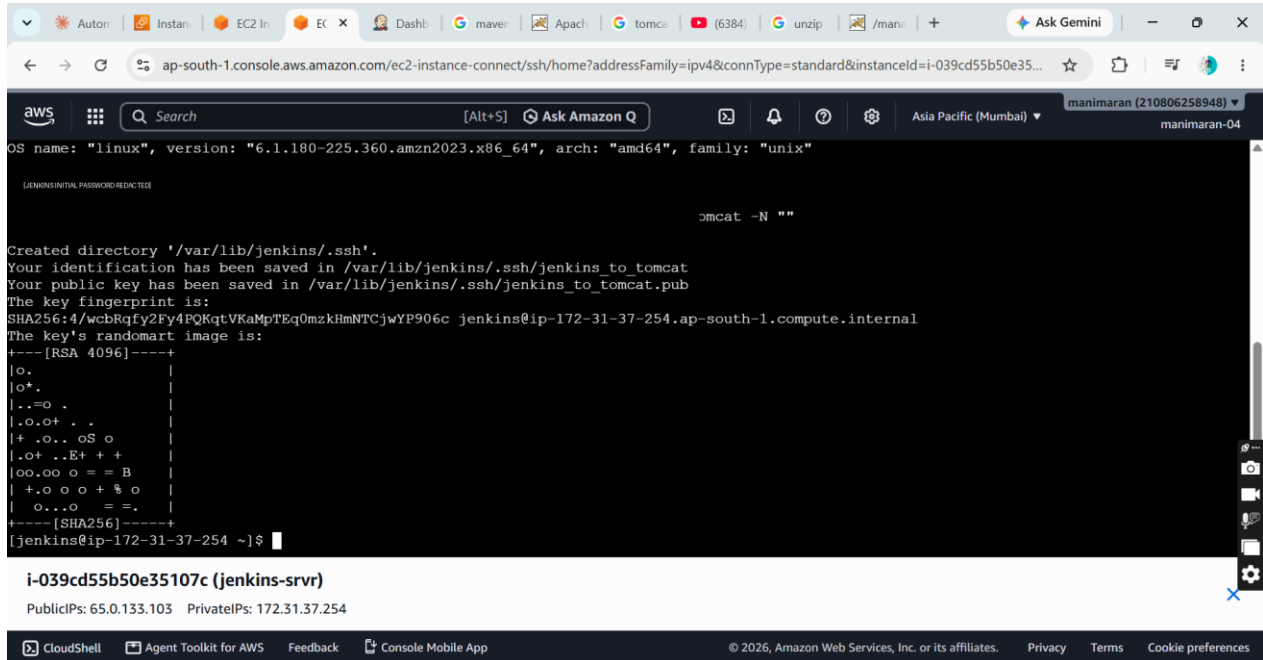
i-044077efaea279d93 (tomcat-srvr)
PublicIPs: 13.201.123.17  PrivateIPs: 172.31.36.131

```

Figure 15. Start Tomcat through systemd and verify service status.

Step 15 – Prepare Jenkins SSH deployment credentials

Instruction / Action: The Jenkins host is accessed and the Jenkins OS user is used to create an SSH key pair for Jenkins-to-Tomcat communication. Sensitive values have been redacted.



```

OS name: "linux", version: "6.1.180-225.360.amzn2023.x86_64", arch: "amd64", family: "unix"

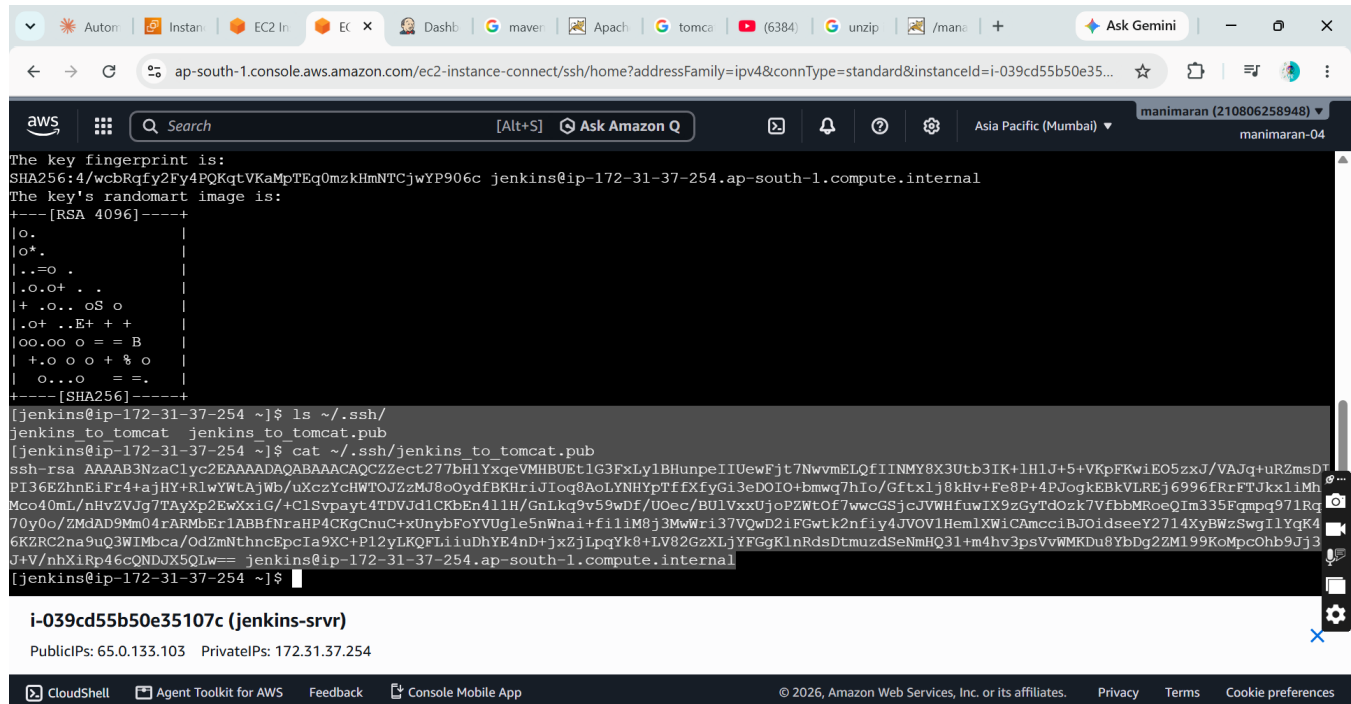
Created directory '/var/lib/jenkins/.ssh'.
Your identification has been saved in /var/lib/jenkins/.ssh/jenkins_to_tomcat
Your public key has been saved in /var/lib/jenkins/.ssh/jenkins_to_tomcat.pub
The key fingerprint is:
SHA256:4/wcbRqfy2Fy4PQKqtVKaMpTEqOmzkHmNtCjwYP906c jenkins@ip-172-31-37-254.ap-south-1.compute.internal
The key's randomart image is:
+---[RSA 4096]-----+
|o.                  |
|o*                 |
|..=O              |
|.o.o+             |
|+ .o.. oS o        |
|.o+ ..E+ + +       |
|oo.oo o = = B      |
|+o o o + % o       |
| o...o = =.        |
+---[SHA256]-----+
[jenkins@ip-172-31-37-254 ~]$

i-039cd55b50e35107c (jenkins-srvr)
PublicIPs: 65.0.133.103 PrivateIPs: 172.31.37.254
  
```

Figure 16. Prepare Jenkins SSH deployment credentials.

Step 16 – Verify the generated SSH key files

Instruction / Action: The Jenkins user verifies the generated SSH key files and displays the public-key material for configuring access on the Tomcat host. The key content is redacted in this documentation copy.



```

The key fingerprint is:
SHA256:4/wcbRqfy2Fy4PQKqtVKaMpTEqOmzkHmNtCjwYP906c jenkins@ip-172-31-37-254.ap-south-1.compute.internal
The key's randomart image is:
+---[RSA 4096]-----+
|o.                  |
|o*                 |
|..=O              |
|.o.o+             |
|+ .o.. oS o        |
|.o+ ..E+ + +       |
|oo.oo o = = B      |
|+o o o + % o       |
| o...o = =.        |
+---[SHA256]-----+
[jenkins@ip-172-31-37-254 ~]$ ls ~/.ssh/
jenkins_to_tomcat jenkins_to_tomcat.pub
[jenkins@ip-172-31-37-254 ~]$ cat ~/.ssh/jenkins_to_tomcat.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQACZect277bH1YxqeVMHBUEt1G3FxLylBhUnpeIIUewFjt7NwvmELQfIINMY8X3Utb3IK+1H1J+5+VKpFKwiEO5zxJ/VAJq+uRZmsDf
PI36EZhEiFr4+aJHY+RlwYtAjb/uXczYchWTOJZzMJ8OydfBKHriJIoq8AoLYNHYPtffXfyGi3eDOIO+bmwg7hIo/Gftxlj8kHv+Fe8P+4PjogKEBkVLRj6996fRrFTJkxliMh
Mco40mL/nHv2VJg7TAyXp2EwXiG/+C1Svpayt4TDVJd1CKbEn411H/GnLkq9v59wDf/UOec/Bu1VxxUjoPZwtOf7wccGSjcJVWHfuwIX9zGyTdOzk7VfbbMRoeQIm335Fqmpq971Rq
70y0o/ZM4AD9Mm04rARMBEr1ABBFnraHP4CKgCnuC+xUnybFoYVUgle5nWnai+filIM8j3MwWri37VQwD2iFGwtk2nfiy4JVOV1HemlXWiCAmccciBJoidseeY2714XyBWzSwgIlYqK4
6KZRC2na9uQ3Wlmbca/OdZmNthncEpcIa9XC+P12yLKQFLiudhYE4nD+jxZjLpYk8+LV82GzXLjYFGgKlnRdsDtmuzdSeNmHQ31+m4hv3psVvWMKDu8YbDg2ZM199K0MpcOhb9Jj3
J+v/nhXiRp46cQNDJX5QLw== jenkins@ip-172-31-37-254.ap-south-1.compute.internal
[jenkins@ip-172-31-37-254 ~]$

i-039cd55b50e35107c (jenkins-srvr)
PublicIPs: 65.0.133.103 PrivateIPs: 172.31.37.254
  
```

Figure 17. Verify the generated SSH key files.

Step 17 – Review Jenkins deployment-related plugins

Instruction / Action: Jenkins Plugin Manager is used to review plugins relevant to Maven integration, SSH-based artifact transfer and deployment to a container.

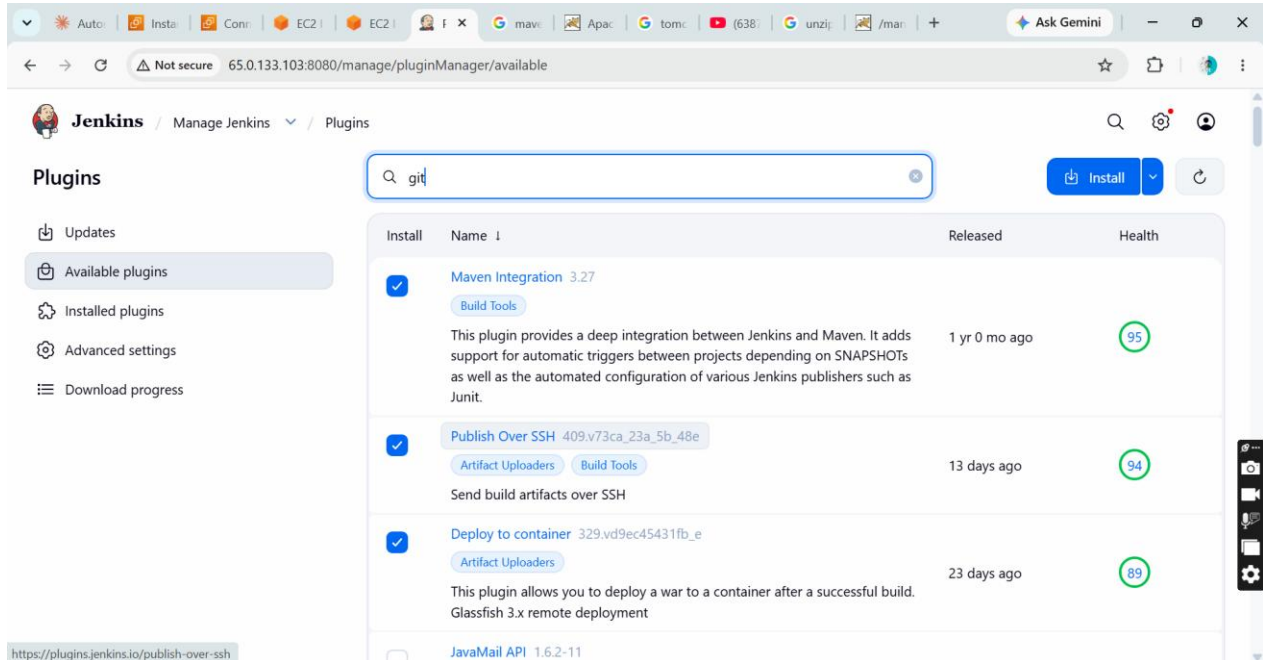


Figure 18. Review Jenkins deployment-related plugins.

Step 18 – Configure Jenkins credentials

Instruction / Action: The Jenkins Credentials page shows credentials configured for GitHub and the Tomcat SSH deployment connection.

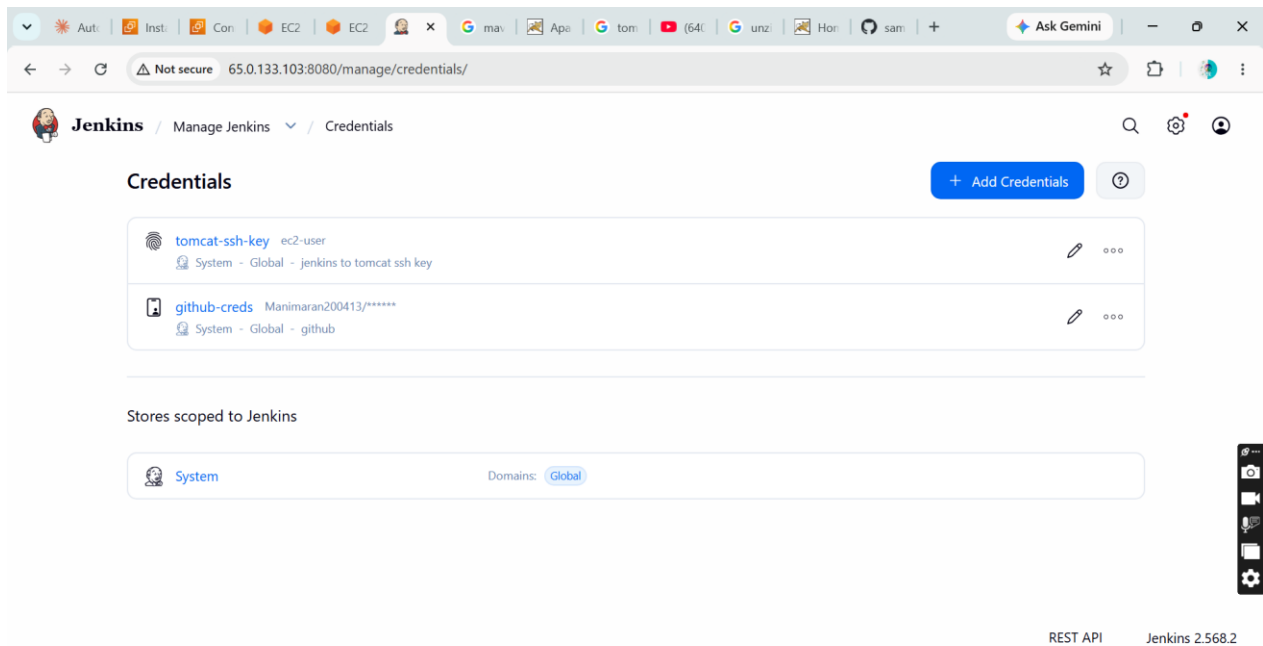


Figure 19. Configure Jenkins credentials.

Step 19 – Verify Jenkins jobs and build history

Instruction / Action: The Jenkins dashboard lists the demo and tomcat_deploy-webapp jobs and displays their recent execution information.

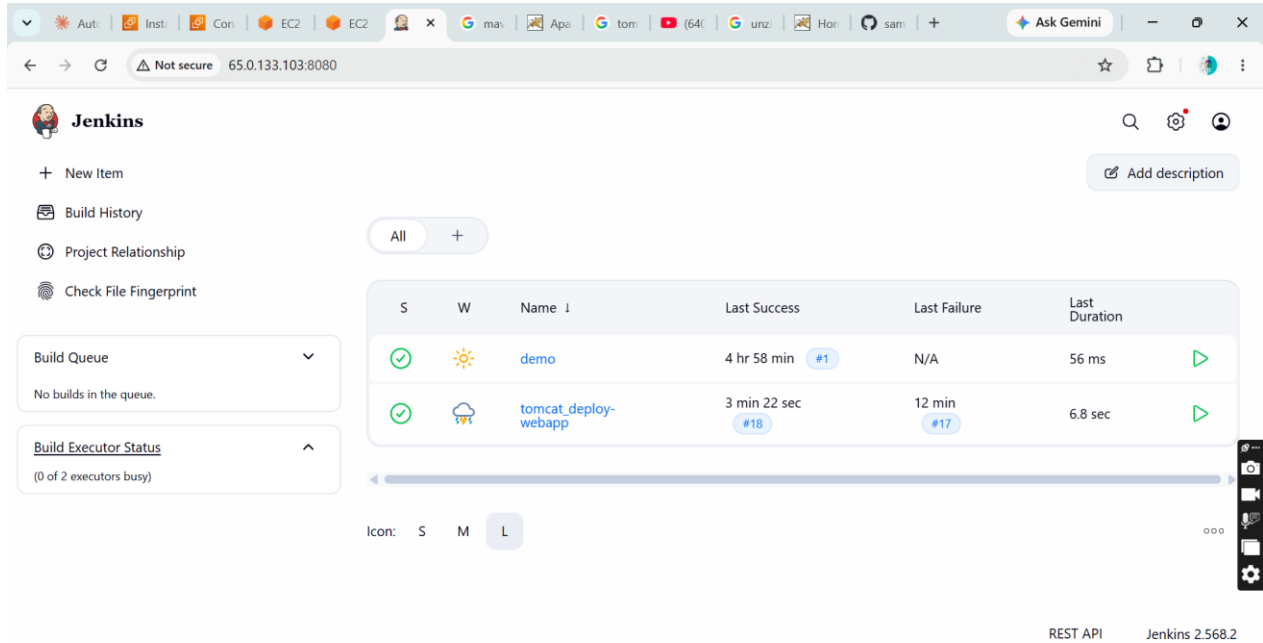


Figure 20. Verify Jenkins jobs and build history.

Step 20 – Review the Jenkins pipeline stages

Instruction / Action: The tomcat_deploy-webapp job displays the pipeline stages: Declarative Tool Install, Checkout, Build and Deploy, together with execution timings.

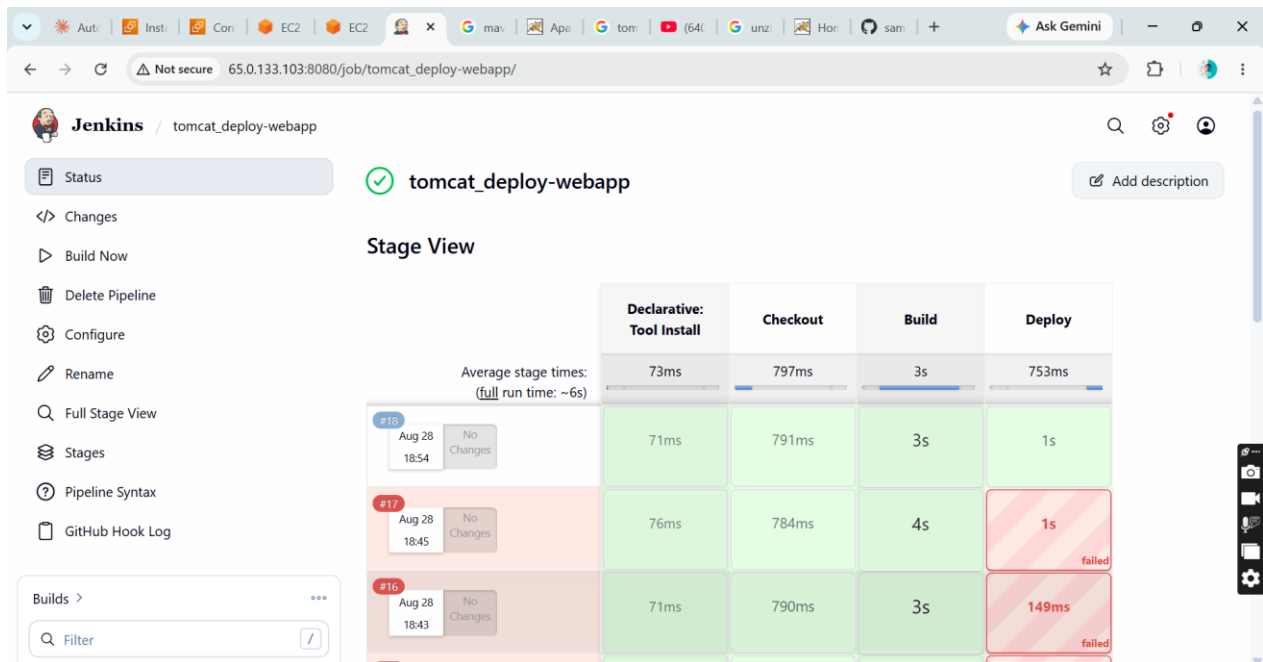


Figure 1. Review the Jenkins pipeline stages.

Step 21 – Verify successful Jenkins pipeline execution

Instruction / Action: The Jenkins console output reaches End of Pipeline and reports Finished: SUCCESS, confirming successful pipeline execution.

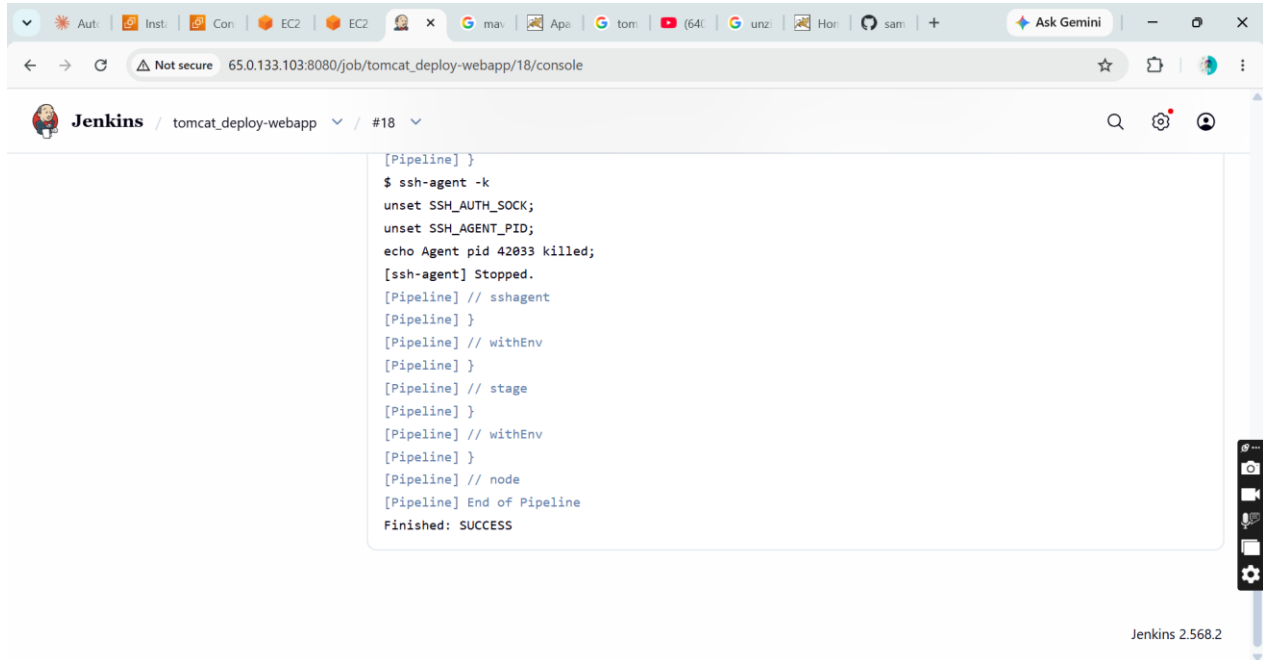


Figure 22. Verify successful Jenkins pipeline execution.

Step 22 – Inspect the Jenkins workspace

Instruction / Action: The Jenkins workspace contains the source directory, target directory, pom.xml and README.md, showing that the project was checked out and built in the workspace.

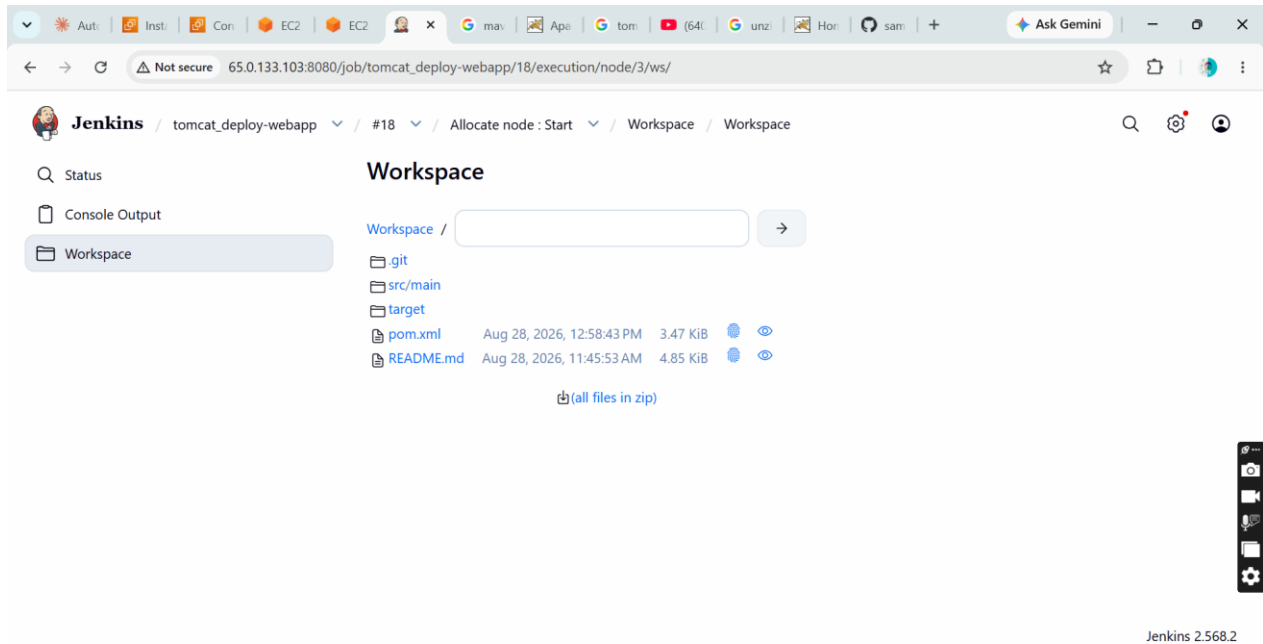


Figure 23. Inspect the Jenkins workspace.

Step 23 – Verify Tomcat ownership, permissions and service state

Instruction / Action: The application server terminal applies Tomcat ownership/permission settings, restarts the service and confirms the running service. The webapps directory contains the deployed sample application.

```

root@ip-172-31-36-131 /]# sudo chown -R tomcat:tomcat /opt/tomcat/apache-tomcat-10.1.59/
root@ip-172-31-36-131 /]# sudo chmod -R 755 /opt/tomcat/apache-tomcat-10.1.59/logs/
root@ip-172-31-36-131 /]# sudo chmod -R 755 /opt/tomcat/apache-tomcat-10.1.59/webapps/
root@ip-172-31-36-131 /]# sudo systemctl reset-failed tomcat
root@ip-172-31-36-131 /]# sudo systemctl restart tomcat
root@ip-172-31-36-131 /]# sudo systemctl status tomcat
● tomcat.service - Apache Tomcat
   Loaded: loaded (/etc/systemd/system/tomcat.service; disabled; preset: disabled)
   Active: active (running) since Fri 2026-08-28 13:24:06 UTC; 16s ago
     Process: 40357 ExecStart=/opt/tomcat/apache-tomcat-10.1.59/bin/startup.sh (code=exited, status=0/SUCCESS)
    Main PID: 40365 (java)
       Tasks: 35 (limit: 4523)
      Memory: 112.1M
         CPU: 3.655s
    CGroup: /system.slice/tomcat.service
            └─40365 /usr/lib/jvm/java-21-amazon-corretto/bin/java -Djava.util.logging.config.file=/opt/tomcat/apache-tomcat-10.1.59/conf/

Aug 28 13:24:06 ip-172-31-36-131.ap-south-1.compute.internal systemd[1]: Starting tomcat.service - Apache Tomcat...
Aug 28 13:24:06 ip-172-31-36-131.ap-south-1.compute.internal startup.sh[40357]: Tomcat started.
Aug 28 13:24:06 ip-172-31-36-131.ap-south-1.compute.internal systemd[1]: Started tomcat.service - Apache Tomcat.
root@ip-172-31-36-131 /]# cd /opt/tomcat/apache-tomcat-10.1.59/webapps
root@ip-172-31-36-131 webapps]# ls
ROOT docs examples host-manager manager sample sample.war
root@ip-172-31-36-131 webapps]#
  
```

i-044077efaea279d93 (tomcat-srvr)
PublicIPs: 13.201.123.17 PrivateIPs: 172.31.36.131

Figure 24. Verify Tomcat ownership, permissions and service state.

Step 24 – Validate the deployed application in Tomcat Manager

Instruction / Action: The Tomcat Web Application Manager lists /sample as a running Sample Web Application, providing application-level deployment validation.

tomcat web Application manager

Message: OK

Manager					
List Applications	HTML Manager Help	Manager Help	Server Status		
Applications					
Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/sample	None specified	Sample Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Figure 25. Validate the deployed application in Tomcat Manager.

Step 25 – Validate the final SampleApp web interface

Instruction / Action: The browser displays the deployed SampleApp interface with Java 17 and Tomcat 10 indicators, confirming successful user-facing application access.

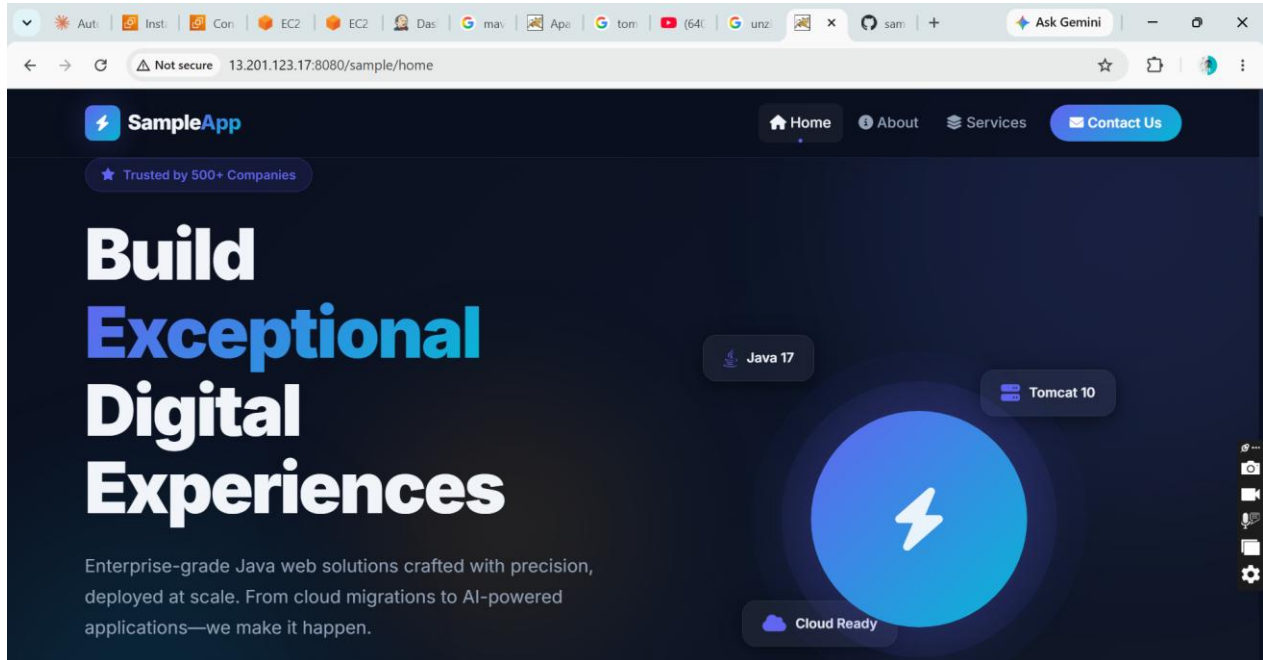


Figure 26. Validate the final SampleApp web interface.

10. Testing and Validation

Validation Area	Observed Evidence
Build environment validation	The terminal evidence shows Git, Java and Maven checks on the Jenkins/build EC2 environment.
Jenkins validation	The Jenkins dashboard and job pages show the tomcat_deploy-webapp job and its pipeline stages.
Tomcat validation	The Tomcat welcome page, Manager page and systemd status demonstrate that Tomcat is reachable and running.
Workspace validation	The Jenkins workspace contains src/main, target, pom.xml and README.md.
Pipeline validation	The Jenkins console ends with “Finished: SUCCESS” for the demonstrated build.
Application validation	The Tomcat Manager lists /sample as a running Sample Web Application, and the browser displays the SampleApp interface.

11. Troubleshooting and Operational Notes

Command syntax: One screenshot records a mistyped Jenkins version option and a Maven command that was not found when using the name “maven”; the subsequent “mvn --version” check confirms Maven is available. This should be treated as command-level troubleshooting rather than a project failure.

Tomcat service: The evidence shows Tomcat being started manually and later managed through a systemd service. Service status output confirms an active Java process after configuration.

Tomcat Manager access: The manager application requires authentication and role configuration. Credentials should be stored securely and should not be committed to source control.

Jenkins deployment connection: SSH key generation and Jenkins credential configuration are shown. If deployment fails, verify the Jenkins credential, remote-user permissions, SSH connectivity, target path and Tomcat permissions.

Application reachability: If the application is not reachable, verify the Tomcat service, EC2 security-group rules, port 8080, deployment path and the running application entry in Tomcat Manager.

12. Security Considerations

- Do not commit passwords, private keys, access tokens or AWS credentials to GitHub.
- Use Jenkins Credentials or another approved secret-management mechanism rather than hard-coding secrets in pipeline files.
- Restrict EC2 security-group rules to only the ports required by the project.
- Use a dedicated deployment user with only the permissions needed for Tomcat deployment where practical.
- Protect the Jenkins administrative interface and Tomcat Manager interface with strong authentication.
- The screenshots embedded in this report have sensitive password/key material redacted.

13. Advantages

- Automates a repeatable Java application build and deployment workflow.
- Reduces repeated manual WAR-copy and server-restart operations.
- Provides visible Jenkins stages and console logs for traceability.
- Separates CI/build responsibility from the Tomcat application server.
- Supports quick validation through both pipeline status and application-level browser testing.

14. Limitations

- The demonstrated deployment uses EC2/Tomcat and does not include container orchestration.
- The architecture exposes the application directly through a Tomcat endpoint on port 8080.
- High availability, load balancing and auto scaling are outside the demonstrated Level 4 scope.
- The supplied evidence does not establish a production-grade secret-management or multi-environment release strategy.

15. Future Enhancements

- Add automated unit and integration test reporting.
- Add Docker containerization and an image registry for a later project level.
- Introduce an Application Load Balancer and Auto Scaling for higher availability.
- Add CloudWatch monitoring, logging and alerting.
- Add separate development, staging and production deployment environments.
- Add approval gates and rollback mechanisms for production releases.

16. Key Lessons Learned

- Jenkins can coordinate source checkout, build and deployment stages in a repeatable workflow.
- Maven provides a standard way to compile, test and package a Java web application as a WAR.
- Tomcat can be managed both with startup scripts and through systemd service configuration.
- SSH-based deployment requires correct key configuration, remote permissions and secure credential handling.
- Successful CI/CD validation should include both Jenkins execution evidence and application-level browser validation.

17. Interview / Viva Explanation

"I implemented a Level 4 CI/CD pipeline for a Java web application using GitHub, Jenkins, Maven and Apache Tomcat on AWS EC2. The developer pushes code to GitHub, which triggers Jenkins through a webhook. Jenkins checks out the project and uses Maven to build and package the application as a WAR file. The deployment stage uses SSH/SCP communication to transfer the application to the Tomcat EC2 server. Tomcat serves the application on port 8080. I verified the workflow using Jenkins stage/console output, Tomcat Manager and the deployed SampleApp browser interface."

18. Conclusion

The Level 4 project demonstrates a practical Java web application CI/CD workflow on AWS. The supplied evidence covers the Jenkins build environment, Java/Maven preparation, Tomcat installation and service management, Tomcat Manager configuration, Jenkins credentials, project workspace, pipeline stages, successful pipeline execution and final browser validation. The resulting delivery path is:

Developer → GitHub → Jenkins → Checkout → Maven → WAR → SSH/SCP → Tomcat EC2 → Tomcat :8080 → Java Web Application

Appendix A — Implementation Evidence Inventory

Figures	Screenshots	Demonstrates
Figure 2	Screenshot 2026-08-28 135400.png	Create Instance & Verify the basic build environment
Figure 3	Screenshot 2026-08-28 145839.png	Verify Jenkins availability and create the pipeline job
Figure 4	Screenshot 2026-08-28 150239.png	Verify Java and Tomcat installation on the application EC2 instance
Figure 5	Screenshot 2026-08-28 150252.png	Verify Tomcat is reachable on port 8080
Figure 6	Screenshot 2026-08-28 150302.png	Attempting to Tomcat Web Application Manager Page
Figure 7	Screenshot 2026-08-28 150835.png	Opening the Context.xml file
Figure 8	Screenshot 2026-08-28 150859.png	Verify the context.xml file
Figure 9	Screenshot 2026-08-28 151200.png	Inspect Tomcat configuration and restart the server
Figure 10	Screenshot 2026-08-28 151230.png	Open the Tomcat Manager application
Figure 11	Screenshot 2026-08-28 151631.png	Configure Tomcat Manager access
Figure 12	Screenshot 2026-08-28 151653.png	Authenticate to Tomcat Manager
Figure 13	Screenshot 2026-08-28 151706.png	Verify the Tomcat Web Application Manager
Figure 14	Screenshot 2026-08-28 152551.png	Prepare the Tomcat systemd service
Figure 15	Screenshot 2026-08-28 155017.png	Start Tomcat through systemd and verify service status
Figure 16	Screenshot 2026-08-28 155338.png	Prepare Jenkins SSH deployment credentials
Figure 17	Screenshot 2026-08-28 155512.png	Verify the generated SSH key files
Figure 18	Screenshot 2026-08-28 163117.png	Review Jenkins deployment-related plugins
Figure 19	Screenshot 2026-08-28 185803.png	Configure Jenkins credentials
Figure 20	Screenshot 2026-08-28 185817.png	Verify Jenkins jobs and build history
Figure 21	Screenshot 2026-08-28 185834.png	Review the Jenkins pipeline stages
Figure 22	Screenshot 2026-08-28 185847.png	Verify successful Jenkins pipeline execution
Figure 23	Screenshot 2026-08-28 185924.png	Inspect the Jenkins workspace
Figure 24	Screenshot 2026-08-28 190130.png	Verify Tomcat ownership, permissions and service state
Figure 25	Screenshot 2026-08-28 190154.png	Validate the deployed application in Tomcat Manager
Figure 26	Screenshot 2026-08-28 190210.png	Validate the final SampleApp web interface

END OF LEVEL 4 PROJECT DOCUMENTATION